



SE618 LOGISTIC CASE STUDY ANALYSIS

WEEK 8 – PROGRESS REPORT

PROJECT TITLE: SERGE – AI Agent for Supply Chain Operations

Prototype Results, Modelisation Refinement & First Evaluations

Gauthier SCHMITT	6822807466
Louis GRIGNOLA	6822807516
Matteo SCHWARZ	6822807557
Maël TELLUS	6822807532

SUBMITTED TO:

AJARN WARUT

MARCH 2026

SEMESTER 2/2025

SCHOOL OF MANUFACTURING SYSTEMS AND MECHANICAL ENGINEERING
SIRINDHORN INTERNATIONAL INSTITUTE OF TECHNOLOGY
THAMMASAT UNIVERSITY

Contents

1	Executive Summary	2
2	Advisor Feedback & Modelisation Refinement	2
2.1	Feedback Received	2
2.2	Changes Implemented	2
3	Supply Chain Modelisation	3
3.1	Business Context: Siam Mango Co.	3
3.2	Entity Model	3
3.2.1	Procurement Domain (SRM)	3
3.2.2	Inventory Domain (WMS)	3
3.2.3	Sales Domain (OMS)	4
3.2.4	Shipping Domain (TMS)	4
3.3	Cross-System Relationships	4
3.4	Data Scale	5
4	Architecture & Implementation	5
4.1	Clean Architecture in Go	5
4.2	MCP Integration	6
4.3	Agent Configuration	6
5	First Results: Agent Performance	6
5.1	Evaluation Framework	6
5.2	Qualitative Results	7
5.3	Detailed Results	7
5.3.1	Cross-System Diagnostic (SHP-009 Carrier Mismatch)	7
5.3.2	Full Supply Chain Tracing (PROD-001 History)	7
5.3.3	Complex Fulfillment Analysis (ORD-028)	8
5.4	Key Observations	8
6	Evaluation Module	8
6.1	Structure	9
6.2	Scenario Format	9
7	Next Steps	10

Executive Summary

This week marks a turning point: SERGE moved from design to **working prototype with measurable results**. The full supply chain was modelled, implemented as a Go monorepo, exposed as 35 MCP tools, seeded with 500 customer orders, and tested with Claude Code as the reasoning agent. Advisor feedback led to a significant modelisation refinement (SupplierProduct entity, explicit naming convention) that improved both code clarity and agent understanding.

The agent successfully answered complex cross-system diagnostic questions across all four supply chain systems (SRM, WMS, OMS, TMS), achieving near-perfect factual accuracy on our evaluation scenarios.

Advisor Feedback & Modelisation Refinement

Feedback Received

Our advisor reviewed the initial prototype and identified three areas for improvement:

1. **Supplier–Product relationship:** The model lacked a proper link between what suppliers sell and what we sell. Suppliers should expose their own products (with their own pricing), and our products reference those.
2. **Price redundancy:** Unit costs and prices were duplicated on order lines. Instead, prices should live on the entities: `SupplierProduct.unit_cost` for purchase cost, `Product.unit_price` for selling price.
3. **Naming clarity:** Terms like “Order” and “PurchaseOrder” were ambiguous. The domain language should be explicit: `CustomerOrder`, `SupplierOrder`, `SupplierProduct`.

Changes Implemented

Table 1: Modelisation changes following advisor feedback.

Before	After	Rationale
Order	CustomerOrder	Explicit: this is a customer’s order
PurchaseOrder	SupplierOrder	Explicit: this is our order to a supplier
OrderLine.unit_price	Removed	Price comes from <code>Product.unit_price</code>
POLine.unit_cost	Removed	Cost comes from <code>SupplierProduct.unit_cost</code>
Product.supplier_ids	Removed	Link goes through <code>SupplierProduct</code>
(none)	SupplierProduct	New entity: what a supplier sells, at what cost

Key Insight

The **SupplierProduct** entity is the bridge between the procurement and inventory domains. A supplier doesn't sell "our" product – they sell their own item (e.g., "Fresh Nam Doc Mai Mango from Chanthaburi" at 280 THB) which feeds into our product catalog ("Nam Doc Mai Box 5kg" sold at 450 THB). This eliminates price redundancy: purchase cost lives on **SupplierProduct**, selling price on **Product**, and order lines only carry quantity.

Supply Chain Modelisation

Business Context: Siam Mango Co.

We model a **small B2B mango distribution company** based in Bangkok, Thailand. The company buys mangoes (fresh and processed) from Thai farms, stores them in a central warehouse, and delivers to B2B customers (hotels, restaurants, supermarkets).

Entity Model

The supply chain is modelled across **four bounded domains**, each corresponding to a real-world enterprise system:

Table 2: Four domains and their entities.

Domain	System	Entities
Procurement	SRM	Supplier, SupplierProduct, SupplierOrder, SupplierOrderLine
Inventory	WMS	Product, Warehouse, InventoryRecord, StockMovement
Sales	OMS	Customer, CustomerOrder, CustomerOrderLine
Shipping	TMS	Carrier, Shipment, TrackingEvent

Procurement Domain (SRM)

Supplier – A Thai mango farm or processor. Key attributes: region, declared lead time, reliability rating (1.0–5.0), product categories, payment terms (**net_15**, **net_30**, **cod**).

SupplierProduct – An item a supplier offers for sale. Carries the **unit_cost** (the price we pay). Links a supplier to one of our products. Multiple suppliers can offer the same product at different costs, enabling cost comparison.

SupplierOrder – A purchase order we place with a supplier to restock. Status flow: **pending** → **confirmed** → **shipped** → **received**. Lines reference SupplierProducts (no price on the line – cost comes from the entity).

Inventory Domain (WMS)

Product – What we sell to customers. Carries the **unit_price** (our selling price). Key attributes for perishable goods: **shelf_life_days** (3–180 days), **storage_type** (**refrigerated** or **ambient**), **reorder_point**.

InventoryRecord – Current stock snapshot: quantity on hand, reserved units, available = quantity – reserved. Alerts when available falls below reorder point.

StockMovement – Audit trail of every stock entry (from SupplierOrders) and exit (from CustomerOrders), with reference traceability.

Sales Domain (OMS)

Customer – B2B buyer with segment (**retail** or **wholesale**), delivery address, and delivery zone (**central_bkk** or **outer_bkk**).

CustomerOrder – Sales order with priority levels: **normal** (5 business days), **high** (2–3 days), **urgent** (ship within 24h). Lines reference Products (no price on the line – price comes from the entity).

Shipping Domain (TMS)

Carrier – Transport company with type (**ground** or **refrigerated**), cost per kg, and average transit time. Critical business rule: products with **storage_type: refrigerated** must use a refrigerated carrier.

Shipment – Physical delivery tied to a CustomerOrder. One order can have multiple shipments (split shipment). Statuses: **pending** → **in_transit** → **delivered** or **exception**.

Cross-System Relationships

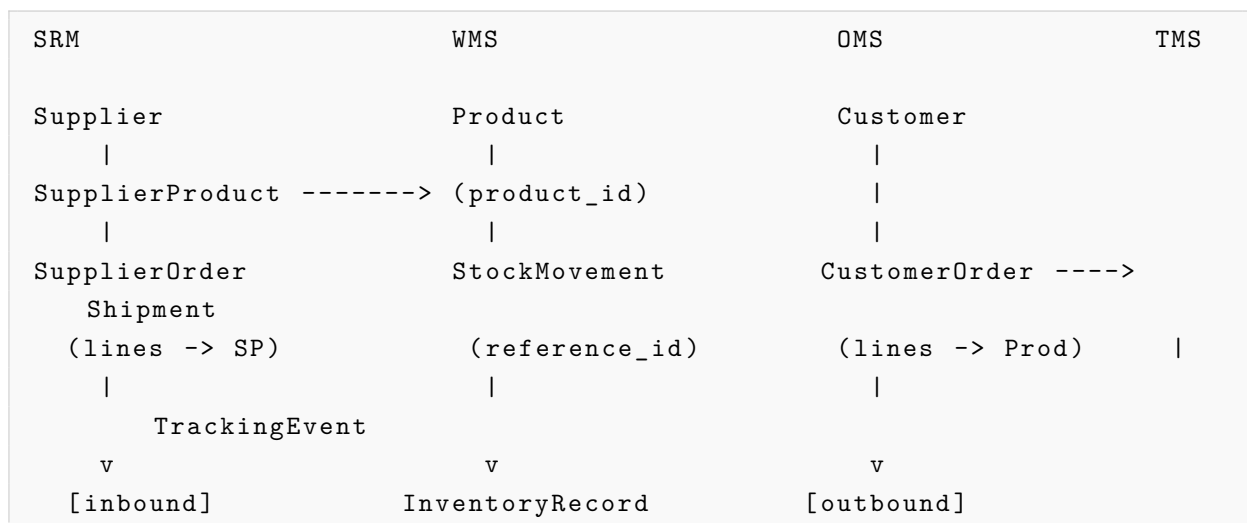


Figure 1: Entity relationships across the four systems. Arrows indicate foreign key references.

Data Scale

Table 3: Seed data volumes (generated with fixed seed for reproducibility).

Entity	Count	Notes
Suppliers	5	Thai farms across 5 regions
Supplier Products	15	Bridge entity: supplier × product
Products	8	5 fresh (refrigerated) + 3 processed (ambient)
Warehouse	1	Bangkok Central Warehouse
Customers	12	Hotels, restaurants, supermarkets
Carriers	3	1 ground + 2 refrigerated
Customer Orders	500	3 months of order history (Oct 2025 – Feb 2026)
Supplier Orders	60	Restocking purchase orders
Shipments	493	Including split shipments
Tracking Events	1,431	3–4 events per shipment
Stock Movements	965	Inbound from POs + outbound from orders

Ground Truth Preservation

The first 30 customer orders (ORD-001 to ORD-030) and first 18 supplier orders (PO-001 to PO-018) are hand-crafted evaluation scenarios with known ground truth. The remaining data (IDs ≥ 100) is generated for volume realism. This separation ensures evaluation ground truth is never affected by data regeneration.

Architecture & Implementation

Clean Architecture in Go

The system follows a strict layered architecture inspired by Domain-Driven Design:

Table 4: Architecture layers and their responsibilities.

Layer	Package	Responsibility
Domain	<code>internal/domain/</code>	Pure structs and business methods. Zero dependencies.
Repository	<code>internal/repository/</code>	In-memory data stores. Loaded from JSON at startup.
Service	<code>internal/service/</code>	Business logic: aggregations, comparisons, computed views.
Controller	<code>internal/controller/</code>	Thin adapter: converts string params to typed calls.
MCP Server	<code>internal/mcpserver/</code>	Registers MCP tools and wires handlers to controllers.

Call flow: `Agent` → `stdio` → `MCP Server` → `Controller` → `Service` → `Repository` → `Domain`

The domain layer is organised by **business language**, not by system name:

Table 5: Domain files use business names; system names appear only at the MCP layer.

Domain File	Business Concept	MCP Tool Prefix
procurement.go	Suppliers, purchasing	srm_*
inventory.go	Products, stock	wms_*
sales.go	Customers, orders	oms_*
shipping.go	Carriers, delivery	tms_*

MCP Integration

The MCP layer wraps controllers as **35 read-only tools** accessible to any MCP-compatible agent via stdio transport. Each tool has a clear description following Anthropic’s Agent-Computer Interface (ACI) guidelines, optimised for LLM comprehension.

Table 6: MCP tools by system (35 total).

System	Tools	Key Capabilities
SRM	11	Get/list suppliers, supplier products, supplier orders; compare suppliers; find overdue POs; list products by supplier or by our product
WMS	8	Get/list products; check inventory; list low stock; get stock movements; full inventory snapshot; warehouse details
OMS	8	Get/list customer orders; search orders by product, customer, or status; list overdue orders; list customers
TMS	6	Get/track shipments; list by status; list exceptions; get/list carriers

Agent Configuration

The agent operates in a dedicated project folder containing:

- **CLAUDE.md** – Business context document that provides the agent with domain knowledge: entity definitions, enum meanings (e.g., “urgent = ship within 24h”), business rules (“refrigerated products must use refrigerated carriers”), and cross-system reasoning patterns.
- **.mcp.json** – Declares the MCP server binary and data path.

First Results: Agent Performance

Evaluation Framework

We designed 15 evaluation scenarios (5 per use case) stored as structured markdown files in `serge/eval/`. Each scenario defines:

- The natural language question
- Expected MCP tool calls (ordered)
- Ground truth answer derived from seed data
- Must-contain and must-not-contain criteria

- Evaluation checklist

Qualitative Results

We tested the agent on four representative queries of increasing complexity. The following table summarises the results:

Table 7: Agent performance on four test queries (scored on 5-point scale).

Query	Systems	Tools	Factual	Complete	Reasoning	Score
Order status (ORD-005)	2	2	5	5	5	5.0
Carrier mismatch (SHP-009)	4	4	5	5	5	4.85
PROD-001 full history	4	6	5	5	5	5.0
ORD-028 fulfillment analysis	4	7–9	5	4–5	5	4.7–5.0

Detailed Results

Cross-System Diagnostic (SHP-009 Carrier Mismatch)

Question: “Shipment SHP-009 had a refrigeration failure but was shipped via Kerry Express (ground carrier). Was this the right carrier choice for perishable mangoes?”

The agent chained 4 tool calls across TMS → OMS → WMS:

1. `tms_get_shipment("SHP-009")` – found exception: refrigeration malfunction
2. `tms_list_carriers()` – compared all carrier options
3. `oms_get_customer_order("ORD-011")` – identified the product (PROD-003)
4. `wms_get_product("PROD-003")` – confirmed: `storage_type=refrigerated`, `shelf_life=10 days`

Result

The agent correctly identified that Kerry Express (ground) was the wrong carrier for refrigerated mangoes, computed shipping costs for all 3 carriers (Kerry: 1,875 THB, SCG Cold Chain: 3,125 THB, Flash Express: 4,375 THB), and recommended SCG Cold Chain as the appropriate choice. It referenced the SHP-009 incident as evidence that ground carriers cannot reliably handle perishable goods.

Full Supply Chain Tracing (PROD-001 History)

Question: “Show me the complete supply chain history for PROD-001 – from purchase orders to inventory to customer orders.”

The agent made 6 calls spanning all 4 systems and produced:

- A supplier comparison table (2 suppliers for PROD-001)
- Chronological purchase order history with on-time status
- Stock movement timeline with running balance

- All customer orders consuming PROD-001
- An ASCII flow diagram: Suppliers → Warehouse → Customers
- Key findings: demand spike analysis, reorder point recommendation, margin calculation

Complex Fulfillment Analysis (ORD-028)

Question: “Order ORD-028 needs 12 units of PROD-001 and 6 units of PROD-004 for The Sukhothai Hotel by Feb 18. Can we fulfill it?”

The agent checked stock for both products, identified a 4-unit shortage on PROD-001, found PO-008 arriving on the same day as the deadline, compared refrigerated carriers with computed costs, and proposed three fulfillment options (partial shipment now, wait for PO, priority allocation). It correctly recommended a partial shipment to maintain the hotel’s weekly standing order.

Key Observations

1. **Cross-system reasoning works.** The agent consistently chains tool calls across 3–4 systems to build complete diagnostic pictures. This is the core value proposition.
2. **The agent goes beyond retrieval.** It computes running balances, calculates shipping costs, identifies demand competition between orders, and makes business recommendations – all from read-only data access.
3. **Business context matters.** The CLAUDE.md file significantly improved tool selection accuracy. Without it, the agent sometimes chose suboptimal tools; with it, tool selection matched our expected call sequences.
4. **Consistency varies.** Running the same query twice can produce different levels of detail. The agent sometimes mentions competing orders (ORD-007 vs ORD-028 for PROD-001 stock) and sometimes doesn’t. This will be quantified in the formal evaluation.
5. **Time savings are dramatic.** A human operator checking 4 separate systems for the ORD-028 analysis would need 15–30 minutes. The agent produces a more thorough analysis in under 60 seconds.

Evaluation Module

Structure

Table 8: 15 evaluation scenarios organised by use case.

UC	Scenario	Difficulty	Key Test
UC-1	01 Happy path (ORD-001)	Easy	Basic 2-system retrieval
	02 In transit (ORD-005)	Easy	Tracking events
	03 Exception (ORD-011)	Medium	Exception handling
	04 Cancelled (ORD-014)	Easy	No shipment exists
	05 Late delivery (ORD-003)	Medium	Date comparison
UC-2	01 Stock shortage (ORD-007)	Hard	4-system causal chain
	02 Supplier delay pattern	Medium	Pattern recognition
	03 Carrier exception (SHP-009)	Medium	Cross-system root cause
	04 Order stuck (ORD-012)	Medium	Negative finding
	05 Operational overview	Hard	Full system scan
UC-3	01 Compare all suppliers	Easy	Structured comparison
	02 Best reliability	Easy	Ranking
	03 Spending by supplier	Medium	Cost aggregation
	04 Fastest lead time	Easy	Metric comparison
	05 Late delivery history	Medium	Pattern analysis

Scenario Format

Each scenario is a markdown file containing:

```
# [Title]
## Use Case
UC-X: [Name] | Difficulty: [easy/medium/hard]
## Question
"[Natural language query]"
## Expected Tool Calls
1. tool(param=value) -- [reason]
## Ground Truth
[Facts from seed data the answer MUST contain]
## Must NOT Contain
[Hallucinations or errors to check for]
## Evaluation Checklist
- [ ] Correct tools called
- [ ] Factual accuracy
- [ ] Completeness
- [ ] Source attribution
- [ ] No hallucination
```

Next Steps

Table 9: Remaining work (2 weeks).

Week	Dates	Deliverables
9	Mar 30 – Apr 6	Implement LLM-as-Judge pipeline (judge prompt, scoring rubric). Run 45 evaluation runs (3 per scenario). Measure all 6 metrics (judge score, accuracy, pass rate, tool calls, time, consistency).
10	Apr 7 – 16	Results analysis and visualisation. Final report. Presentation and demo preparation.